ACADEMIC RESEARCH CASE STUDY • ACTIVE LINE-BY-LINE CODE ANALYSIS

Deep Line-by-Line Comparative Analysis & Pattern Recognition in Human vs. AI Code Synthesis: A Large-Scale Empirical Study of 480,000 Code Snippets

Hassan Elkady — Computer Engineering Student, Arab Academy for Science, Technology and Maritime Transport (AAST)

August 2026 | Zenodo Dataset (DOI: 10.5281/zenodo.15423067): 120,000 Tasks / 480,000 Programs Evaluated

Abstract

Evaluating Large Language Model (LLM) code generation requires moving beyond high-level statistical metrics to perform an active, line-by-line comparative analysis of synthesized source code. This paper presents an exhaustive qualitative and quantitative line-by-line investigation analyzing **480,000 code snippets** across **120,000 problem tasks** (60,000 Python and 60,000 Java tasks) from the Zenodo Large-Scale Dataset (10.5281/zenodo.15423067).

1. Universal AI Code Patterns (General AI Fingerprints)

Pattern 1: Step-by-Step Procedural Comment Headers: ChatGPT and Qwen-Coder frequently prefix logic sections with numbered procedural comment headers (# Step 1: Initialize variables, # Step 2: Loop through items).
Pattern 2: Multi-Line Temporary Staging vs Pythonic Unpacking: DeepSeek-Coder and ChatGPT frequently use imperative temporary variables (temp = a; a = b; b = temp), whereas Human Python developers use pythonic tuple unpacking (a, b = b, a).
Pattern 3: Vertical Airiness & Blank Line Padding: ChatGPT and DeepSeek-Coder pad every control statement with empty blank lines (16% - 20% vertical whitespace), whereas Human code is vertically dense (0.3% - 3.4% blank lines).
Pattern 4: Hyper-Enforced PEP-8 Naming: ChatGPT enforces 91.05% snake_case in Python and 96.76% camelCase in Java, suppressing single-character loop variables in favor of verbose descriptive names.

2. Side-by-Side Code Quadruplet Feature Matrix

Code Dimension	Senior Human Developer	OpenAI ChatGPT	DeepSeek-Coder	Alibaba Qwen-Coder
Vertical Layout	Extremely dense (0.3% - 3.4% blank lines)	Air-padded spacing (16% - 20% blank lines)	Moderately spaced (12% - 15% blank lines)	Dense spacing (3% - 4% blank lines)
Documentation	Minimal or none (0% - 4.6% comment density)	Explanatory inline comments (5% - 8%)	Formal docstrings in 55% of Python functions	High inline procedural comments (17% in Java)
Variable Naming	Concise, uses single-letters i,j,k in 30% of code	Verbose descriptive names, PEP-8 hyper-pure	Moderately descriptive names	Concise names, PEP-8 compliant
Control Flow	Complex nested if/else (CC = 3.9 - 4.1)	Flatter guard clauses (CC = 2.5 - 2.7)	Very flat execution flow (CC = 2.1 - 2.5)	Flatter execution flow (CC = 2.1 - 3.2)
Security Risk	Low command injection flaw rate (0.12%)	Higher command injection rate (0.96%, shell=True)	Higher hardcoded secrets rate (0.46% in Java)	High stub retention (25.8% pass/TODO)

3. Real Side-by-Side Code Breakdowns

Refer to `line_by_line_pattern_analysis.md` for full raw side-by-side code blocks across Python and Java task quadruplets.